

Technical Reference Guide: The Finite-Difference Time-Domain (FDTD) Method

Deconstructing Space-Time Discretization, Yee Grid Formulations, and Solver Algorithms

Prepared for the MATLAB Electromagnetic App Project

September 3, 2026

Abstract

This technical reference guide provides a comprehensive mathematical and algorithmic overview of the Finite-Difference Time-Domain (FDTD) method, originally developed by Kane Yee in 1966. It serves as a pedagogical and technical companion to the MATLAB App FDTD simulation software. The guide deconstructs continuous Maxwell's equations into space-time discrete central differences, details the spatial staggering of the Yee Grid, derives the explicit 1D and 2D TM_z update formulations, outlines practical boundary conditions (Mur Absorbing Boundary Conditions), and explains the execution of real-time spectral analysis via Discrete Fourier Transforms (DFT).

1 Introduction to the FDTD Method

The Finite-Difference Time-Domain (FDTD) method is an elegant, state-of-the-art numerical analysis technique used to model electrodynamics. Specifically, it simulates how electromagnetic waves propagate through space, interact with various materials, and scatter or transmit.

FDTD breaks down continuous physical phenomena into discrete computational blocks. To understand the term, we can deconstruct its name:

Finite-Difference (FD): Instead of dealing with continuous derivatives and differential calculus, we approximate spatial and temporal derivatives using algebraic differences on a discrete grid. We replace derivatives in Maxwell's equations with central differences.

Time-Domain (TD): The wave equations are solved step-by-step in time. This is equivalent to producing a "stop-motion movie" or a sequence of simulation frames showing exactly how the fields propagate, oscillate, and evolve second-by-second.

The central mathematical engine behind this methodology is the **Yee Algorithm**, developed by Kane S. Yee in 1966. It relies on a 4-step framework:

1. Discretize Maxwell's curl equations.
2. Discretize continuous physical space and time onto a grid.

3. Find algebraic update equations that solve for future field values using current and past values.
4. Iteratively calculate Electric (E) and Magnetic (H) fields in an interleaved, "leapfrog" manner.

2 Deconstructing Maxwell's Equations

Electromagnetic wave propagation is fundamentally governed by **Maxwell's curl equations**. In a source-free, isotropic, and physical medium, these equations are:

$$\nabla \times \mathbf{E} = -\frac{\partial \mathbf{B}}{\partial t} = -\mu \frac{\partial \mathbf{H}}{\partial t} \quad (1)$$

$$\nabla \times \mathbf{H} = \frac{\partial \mathbf{D}}{\partial t} = \epsilon \frac{\partial \mathbf{E}}{\partial t} \quad (2)$$

where $\mathbf{E}$ is the electric field vector, $\mathbf{H}$ is the magnetic field vector, ϵ is the material permittivity ($\epsilon = \epsilon_0 \epsilon_r$), and μ is the material permeability ($\mu = \mu_0 \mu_r$). In free space, $\epsilon = \epsilon_0$ and $\mu = \mu_0$.

To solve these equations numerically, we simplify them based on polarization and spatial dimensions.

2.1 1D Wave Propagation

For a 1D transverse electromagnetic plane wave propagating along the z -direction, the electric and magnetic fields are orthogonal to each other and perpendicular to the direction of travel. Let us choose a configuration where:

$$\mathbf{E} = E_y(z, t) \hat{\mathbf{y}}, \quad \mathbf{H} = H_x(z, t) \hat{\mathbf{x}} \quad (3)$$

Under this polarization, Maxwell's curl equations simplify to two coupled first-order partial differential equations:

$$\frac{\partial H_x}{\partial t} = \frac{1}{\mu} \frac{\partial E_y}{\partial z} \quad (4)$$

$$\frac{\partial E_y}{\partial t} = \frac{1}{\epsilon} \frac{\partial H_x}{\partial z} \quad (5)$$

Here, the spatial derivative of the electric field drives the time-evolution of the magnetic field, and the spatial derivative of the magnetic field drives the time-evolution of the electric field.

2.2 2D TM_z Polarization

In two dimensions (xy -plane), waves can split into two independent polarizations: Transverse Magnetic (TM_z) and Transverse Electric (TE_z). For the TM_z mode, the electric field is oriented solely along the z -axis, and the magnetic field lies in the xy -plane:

$$\mathbf{E} = E_z(x, y, t) \hat{\mathbf{z}}, \quad \mathbf{H} = H_x(x, y, t) \hat{\mathbf{x}} + H_y(x, y, t) \hat{\mathbf{y}} \quad (6)$$

Expanding the curl operators, Maxwell's equations resolve into three coupled scalar update equations:

$$\frac{\partial H_x}{\partial t} = -\frac{1}{\mu} \frac{\partial E_z}{\partial y} \quad (7)$$

$$\frac{\partial H_y}{\partial t} = \frac{1}{\mu} \frac{\partial E_z}{\partial x} \quad (8)$$

$$\frac{\partial E_z}{\partial t} = \frac{1}{\epsilon} \left(\frac{\partial H_y}{\partial x} - \frac{\partial H_x}{\partial y} \right) \quad (9)$$

3 The Discretization Scheme: The Yee Grid

To transition from smooth, continuous calculus to a discrete numerical model, FDTD partitions physical space and time into small increments.

Spatial Step ($\Delta x, \Delta y, \Delta z$): The physical domain is chopped into tiny grid cells. The spatial grid spacing must be chosen small enough—typically a fraction of the shortest simulated wavelength (e.g., $\Delta z \approx \lambda_{\min}/50$)—to prevent wave distortion and resolve material interfaces.

Temporal Step (Δt): Time is chopped into small intervals. At each step, we update the state of the wave throughout the entire grid.

The Yee Grid Staggering Principle

FDTD does not compute the Electric and Magnetic fields at the same location or at the same time. Instead, it positions them on a staggered spatial and temporal grid, offset by half-steps:

- **Spatial Offset:** In space, the E and H fields are offset by $\Delta/2$. An electric field point is surrounded by magnetic field points, and vice-versa.
- **Temporal Offset:** In time, E and H are offset by $\Delta t/2$. We compute the magnetic fields at times $t = (k + 1/2)\Delta t$, and the electric fields at times $t = k\Delta t$.

This staggering allows us to write second-order accurate central difference approximations.

3.1 Central Difference Approximation

In calculus, the limit definition of a derivative is evaluated as $\Delta \rightarrow 0$. In FDTD, we approximate derivatives with central differences over a finite step size:

$$\left. \frac{\partial f(w)}{\partial w} \right|_{w_0} \approx \frac{f(w_0 + \Delta/2) - f(w_0 - \Delta/2)}{\Delta} \quad (10)$$

Applying this to a spatial derivative of $E_y(z)$ around a grid node n :

$$\left. \frac{\partial E_y}{\partial z} \right|_n \approx \frac{E_y(n + 1/2) - E_y(n - 1/2)}{\Delta z} \quad (11)$$

Similarly, a temporal derivative of $H_x(t)$ around time step k is approximated by:

$$\left. \frac{\partial H_x}{\partial t} \right|_k \approx \frac{H_x(k + 1/2) - H_x(k - 1/2)}{\Delta t} \quad (12)$$

4 Deriving the FDTD Update Equations

By applying central difference approximations to Maxwell's curl equations, we algebraically rearrange the terms to solve for future values of the fields based on current and past values.

4.1 1D Mathematical Formulation

Let us write the fields on a spatial grid where n represents the spatial index along z , and k represents the time index. Staggering spatial grids such that E_y is defined at integer nodes n and H_x at half-integer nodes $n + 1/2$, we obtain:

$$\frac{H_x^{k+1/2}(n + 1/2) - H_x^{k-1/2}(n + 1/2)}{\Delta t} = \frac{1}{\mu} \left(\frac{E_y^k(n + 1) - E_y^k(n)}{\Delta z} \right) \quad (13)$$

$$\frac{E_y^{k+1}(n) - E_y^k(n)}{\Delta t} = \frac{1}{\epsilon} \left(\frac{H_x^{k+1/2}(n + 1/2) - H_x^{k+1/2}(n - 1/2)}{\Delta z} \right) \quad (14)$$

Rearranging these equations algebraically to solve for the newest values (at time step $k + 1/2$ and $k + 1$ respectively), we derive the FDTD update equations:

$$H_x^{k+1/2}(n + 1/2) = H_x^{k-1/2}(n + 1/2) + \left(\frac{\Delta t}{\mu \Delta z} \right) [E_y^k(n + 1) - E_y^k(n)] \quad (15)$$

$$E_y^{k+1}(n) = E_y^k(n) + \left(\frac{\Delta t}{\epsilon \Delta z} \right) [H_x^{k+1/2}(n + 1/2) - H_x^{k+1/2}(n - 1/2)] \quad (16)$$

In our MATLAB FDTD implementation, the scaling coefficients are simplified using the speed of light in vacuum $c_0 = 3 \times 10^8$ m/s:

$$m_{Ey}(n) = \frac{c_0 \Delta t}{\epsilon_r(n)}, \quad m_{Hx}(n) = \frac{c_0 \Delta t}{\mu_r(n)} \quad (17)$$

Which simplifies the update loop equations into:

$$H_x(n) \leftarrow H_x(n) + \frac{m_{Hx}(n)}{\Delta z} [E_y(n + 1) - E_y(n)] \quad (18)$$

$$E_y(n) \leftarrow E_y(n) + \frac{m_{Ey}(n)}{\Delta z} [H_x(n) - H_x(n - 1)] \quad (19)$$

4.2 2D TM_z Mathematical Formulation

In 2D, we designate E_z on integer coordinate nodes (i, j) , H_x at $(i, j + 1/2)$, and H_y at $(i + 1/2, j)$. The resulting discrete updates are:

$$H_x^{k+1/2}(i, j) = H_x^{k-1/2}(i, j) - \left(\frac{c_0 \Delta t}{\Delta y} \right) [E_z^k(i, j + 1) - E_z^k(i, j)] \quad (20)$$

$$H_y^{k+1/2}(i, j) = H_y^{k-1/2}(i, j) + \left(\frac{c_0 \Delta t}{\Delta x} \right) [E_z^k(i + 1, j) - E_z^k(i, j)] \quad (21)$$

$$E_z^{k+1}(i, j) = E_z^k(i, j) + \frac{c_0 \Delta t}{\epsilon_r(i, j)} \left(\frac{H_y^{k+1/2}(i, j) - H_y^{k+1/2}(i - 1, j)}{\Delta x} - \frac{H_x^{k+1/2}(i, j) - H_x^{k+1/2}(i, j - 1)}{\Delta y} \right) \quad (22)$$

These map directly onto the MATLAB matrix implementation for fast vectorized calculations:

```
% Update Magnetic Fields Hx and Hy
Hx(:, 1:Ny-1) = Hx(:, 1:Ny-1) - (mHx / dy) * (Ez(:, 2:Ny) - Ez(:, 1:Ny-1))
;
Hy(1:Nx-1, :) = Hy(1:Nx-1, :) + (mHy / dx) * (Ez(2:Nx, :) - Ez(1:Nx-1, :))
;

% Update Electric Field Ez (using element-wise matrix operations)
Ez(2:Nx-1, 2:Ny-1) = Ez(2:Nx-1, 2:Ny-1) + ...
    (mEz(2:Nx-1, 2:Ny-1) ./ dx) .* (Hy(2:Nx-1, 2:Ny-1) - Hy(1:Nx-2, 2:Ny-1)) - ...
    (mEz(2:Nx-1, 2:Ny-1) ./ dy) .* (Hx(2:Nx-1, 2:Ny-1) - Hx(2:Nx-1, 1:Ny-2));
```

Listing 1: Vectorized 2D FDTD Field Updates in MATLAB

5 Boundary Conditions and Numerical Stability

5.1 The Courant-Friedrichs-Lewy (CFL) Stability Criterion

If the numerical time step Δt is too large relative to the spatial grid spacing, the FDTD calculation becomes unstable and field magnitudes explode exponentially. To maintain numerical stability, the time step must satisfy the **CFL criterion**:

$$\Delta t \leq \frac{1}{c_0 \sqrt{\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2}}} \quad (23)$$

Physically, this states that the numerical wave cannot propagate across a single grid cell faster than the physical speed of light in vacuum. In our 2D code, we set Δt exactly to this limit to maximize phase velocity accuracy:

$$\Delta t = \frac{1}{c_0 \sqrt{\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2}}} \quad (24)$$

5.2 Mur Absorbing Boundary Conditions (ABC)

Since a computer has finite memory, the simulation grid must terminate at boundaries (e.g., $i = 1, i = N_x$). If left untreated, waves reaching these outer boundaries will bounce off the hard grid edges as if hitting a perfect mirror, reflecting non-physical energy back into the simulation and ruining the experiment.

To simulate an infinite, open free space, we apply a first-order **Mur Absorbing Boundary Condition (ABC)** at the edges. The boundary values are calculated by keeping track of the adjacent inner values from the previous step:

$$E_z^{k+1}(1, j) = E_z^k(2, j) \quad (25)$$

This allows normally-incident plane waves to exit the grid seamlessly without reflecting. In the MATLAB code, this is implemented as:

```
% Save outer edges before main update loop
Ez_left    = Ez(2, :);    Ez_right   = Ez(Nx-1, :);
Ez_bottom  = Ez(:, 2);    Ez_top     = Ez(:, Ny-1);
```

```
% ... update fields inside the grid ...

% Apply boundary condition at outermost nodes
Ez(1, :) = Ez_left;   Ez(Nx, :) = Ez_right;
Ez(:, 1) = Ez_bottom; Ez(:, Ny) = Ez_top;
```

Listing 2: First-Order Mur ABC Implementation

6 Source Injection and Spectral Analysis

6.1 Gaussian Pulse Source

To inject electromagnetic energy into our simulation, we overwrite specific nodes with a source waveform. In our MATLAB project, we use a **temporal Gaussian pulse**:

$$E_{\text{src}}(t) = \exp\left(-\left(\frac{t - t_0}{0.2\tau}\right)^2\right) \quad (26)$$

where $\tau = 0.5/f_{\text{max}}$ dictates the pulse width based on the maximum frequency of interest, and t_0 is a time delay ensuring the pulse starts smoothly near zero. This wideband source introduces a range of frequencies (from 0 to 1 GHz) simultaneously.

6.2 Discrete Fourier Transform (DFT) for Spectral Metrics

To determine the frequency response (i.e., how a material behaves at 1 GHz vs 500 MHz) without running hundreds of separate single-frequency simulations, we compute an on-the-fly **Discrete Fourier Transform (DFT)**.

At each time step T , we accumulate a complex running sum for reflectance, transmittance, and source reference across our frequency array:

$$\text{ref}(f) = \sum_{T=1}^{\text{STEPS}} E_{\text{reflected}}(T) \cdot e^{-j2\pi f T \Delta t} \quad (27)$$

$$\text{trn}(f) = \sum_{T=1}^{\text{STEPS}} E_{\text{transmitted}}(T) \cdot e^{-j2\pi f T \Delta t} \quad (28)$$

$$\text{src}(f) = \sum_{T=1}^{\text{STEPS}} E_{\text{src}}(T) \cdot e^{-j2\pi f T \Delta t} \quad (29)$$

Once the time-stepping finishes, the software normalizes these values by the source reference spectrum to compute the exact spectral power coefficients:

Reflectance (R): The percentage of wave energy bouncing backward:

$$R(f) = \left| \frac{\text{ref}(f)}{\text{src}(f)} \right|^2 \quad (30)$$

Transmittance (T): The percentage of wave energy passing forward:

$$T(f) = \left| \frac{\text{trn}(f)}{\text{src}(f)} \right|^2 \quad (31)$$

Conservation of Energy: For lossless media, the sum of Reflectance and Transmittance must satisfy:

$$R(f) + T(f) = 1.0 \quad (32)$$

7 Summary of the FDTD Algorithm Workflow

The general execution sequence of the FDTD solver is structured as follows:

1. Initialization:

- Define spatial parameters ($\Delta x, \Delta y$), grid size (N_x, N_y), and compute stable Δt .
- Initialize field matrices (E_z, H_x, H_y) and boundary arrays to zero.
- Define material parameters (ϵ_r, μ_r) across the spatial grid.
- Initialize spectral accumulator vectors (ref, trn, src).

2. Main Time-Stepping Loop ($T = 1 \dots \text{STEPS}$):

- **Step 2.1:** Save boundary edges for the absorbing boundary condition.
- **Step 2.2:** Update Magnetic Fields (H_x, H_y) using spatial derivatives of E_z .
- **Step 2.3:** Inject the source waveform (E_{src}) into the designated source node.
- **Step 2.4:** Update Electric Field (E_z) using spatial derivatives of the new H_x, H_y .
- **Step 2.5:** Apply Mur ABC on the outer edges of the grid using the saved boundary edges.
- **Step 2.6:** Continuous DFT Spectral Accumulation:

$$\text{ref}(f) \leftarrow \text{ref}(f) + E_z(\text{sensor_ref}) \cdot e^{-j2\pi f T \Delta t}$$

$$\text{trn}(f) \leftarrow \text{trn}(f) + E_z(\text{sensor_trn}) \cdot e^{-j2\pi f T \Delta t}$$

$$\text{src}(f) \leftarrow \text{src}(f) + E_{\text{src}}(T) \cdot e^{-j2\pi f T \Delta t}$$

- **Step 2.7:** Render wave graphics to the UI (time-domain wavefronts and current frequency-domain reflectance/transmittance plots).

3. Post-Processing and Export:

- Normalize DFT accumulations to get final $R(f)$ and $T(f)$ spectra.
- Export the final calculated frequency, reflectance, transmittance, and conservation curve to a timestamped Excel file.

8 Analytical Case Study: Predefined Simulation Media

The MATLAB App defines physical materials by assigning permittivity and permeability scaling values.

Multi-layer devices are modeled as consecutive layers of different materials, creating interesting constructive and destructive spectral resonance profiles which the on-the-fly DFT displays in the App's second plot panel in real-time.

Table 1: Predefined Simulation Materials and Multi-Layer Devices

Material	Relative Permittivity (ϵ_r)	Relative Permeability (μ_r)	Wave Speed (v)	
Air	1.0	1.0	c_0	Fast
Water	80.2	1.0	$0.11c_0$	Extremely Slow
Wood	3.0	1.0	$0.58c_0$	Medium
Glass	4.7	1.0	$0.46c_0$	Relatively Slow
Stainless Steel	6.0	40.0	$0.06c_0$	Very Slow
Graphite	12.0	10.0	$0.09c_0$	Slow
Device 1 (Al-Si-Gl)	4.5, 3.7, 4.7	1.0, 1.0, 1.0	Multi-valued	Medium
Device 2 (SS-Gr-P)	6.0, 12.0, 2.0	40.0, 10.0, 1.0	Multi-valued	Highly Variable